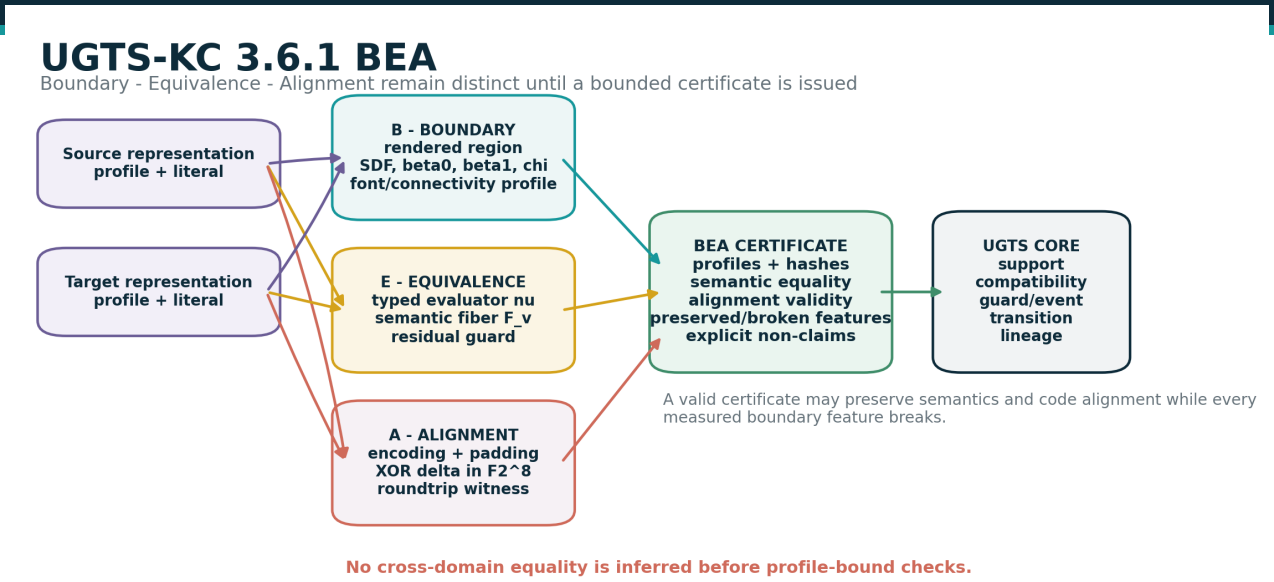
UNIFIED GEOMETRIC-TOPOLOGICAL SUBSTRATE

UGTS-KC 3.6.1 – BEA

Boundary - Equivalence - Alignment

A formal representational hinge calculus distilled from the attached document



Version	3.6.1, 17 August 2026
Version title	BEA. Expanded in this specification as **Boundary-Equivalence-Alignment**; this expansion is engineering-derived.
Requester attribution	Tom Klootwijk; NL200678942; 10-07-1990. Supplied by the requester and not independently verified.
Technical scope	32 BEA delta operators, 16 reusable recipes, literal JSON Schema/example, exact XOR witness, four-font glyph-topology stress data, dependency-free Python core and 66 passing tests.
Evidence rule	Exact source arithmetic is retained; profile-bound observations are typed; invalid topology, entropy, SDF and universality escalations are corrected or rejected.

Abstract

UGTS-KC 3.6.1 adds a formal comparison layer for representations. The attached document moves among a proposed phonetic alias, alphabet-position geometry, rendered text holes, leet rewriting, ASCII bit patterns and a common numerical value. The reusable mathematical trick is not that those objects are identical. It is that they can be connected by explicit maps while their domains remain separate.

The requested version title is **BEA**. This document expands it as **Boundary-Equivalence-Alignment**. The boundary layer measures rendered geometry and planar topology under an explicit font and connectivity profile. The equivalence layer places representations in a common typed semantic fiber. The alignment layer gives a reversible byte witness under a declared encoding, alignment and padding rule. A BEA certificate records all three, names the features that are preserved or broken, and carries explicit non-claims.

The source's bytewise table is reproduced exactly after correcting the term "bit shift" to "padded XOR alignment." For the supplied leet pair, source and target Hamming weights are 39 and 37, while the XOR delta has weight 38. Global parity is preserved, but Hamming weight and entropy are not. Repeated mask `0x5d` is explained by an adjacent swap; repeated mask `0x5e` is explained by an affine parallelogram identity in $\mathbb{F}_2^8$.

The source's font-free hole invariant is not reproduced. Under the shipped DejaVu Sans profile, *negentien* has $(\beta_0, \beta_1, \chi) = (10, 4, 6)$ and *no neg moat* has $(9, 5, 4)$. Lato changes both values again. Token count is kept as a separate lexical feature. A valid BEA certificate therefore binds the pair only at the bounded level *semantic-equivalence-plus-code-alignment*; it explicitly does not claim homeomorphism, homotopy equivalence, standard linguistic equivalence or a physical conservation law.

DECISION

Version 3.6.1 retains the 3.6 query-first substrate and inserts a careful representational certificate before cross-domain claims are promoted. Geometry, topology, semantics and code remain typed. Matching one feature never silently upgrades to full equivalence.

Contents

1 Version decision and scope

1.1 Why BEA

The user requested *BEA* as the title. The source itself does not supply a technical expansion of those letters. The expansion *Boundary-Equivalence-Alignment* is chosen because it directly names the three separable mathematical operations hidden in the source:

1. construct or measure the boundary geometry/topology of a representation;
2. evaluate whether two representations map to the same typed semantic value;
3. construct an exact alignment witness in an encoding space.

The title expansion is therefore an engineering normalization, not a quotation or historical claim.

1.2 Increment from 3.6.0

The 3.6.0 package made definitions first-class, content-addressed substrate records. Version 3.6.1 preserves that architecture and adds:

- representation profiles with normalization, transduction, rendering, encoding and semantic evaluators;
- profile-bound planar boundary signatures (β_0, β_1, χ);
- separate lexical token topology and rendered glyph topology;
- exact padded XOR alignment over $\mathbb{F}_2^8$;
- swap, parallelogram and parity lemmas for repeated masks;
- semantic fibers and residual guards distinct from glyph signed-distance fields;
- a BEA hinge certificate with preserved features, broken features and non-claims;
- cross-font, cross-encoding and cross-profile falsification rules.

1.3 Deliverables

The accompanying ZIP contains this PDF and editable source, the complete 3.6 baseline package, the 3.6.1 JSON Schema and literal example, source-normalization notes, exact XOR rows, four-font glyph topology measurements and masks, 32 BEA delta operators, 16 pattern recipes, claims ledger, Python implementation, optional analysis tools and 66 passing tests.

BOUNDARY

The document is a formal technical specification and reproducible analysis. It does not independently validate the source-proposed phonetic alias, prove linguistic uniqueness, establish authorship priority or create a physical law from representational coincidences.

2 Source distillation without contextual distraction

2.1 What the attached document actually contributes

The source proposes a sequence of transformations: a phonetic alias, alphabet-index geometry, counters/holes in letters, leet substitution, an ASCII XOR table, a "topological hinge" and cross-language tests. The technical extraction keeps only operations that can be typed, measured or falsified.

Source motif	Distilled mechanism	Correction or bound
Proposed phonetic alias (lines 1–13)	Versioned alias/transduction profile between literal representations.	The source does not establish a standard Dutch dialect rule; status remains source-proposed.
Alphabet positions (lines 15–20)	Sequence path $p_i = (i, \psi(c_i))$ with measurable bounding box, centroid, variance and length.	Coordinates come from the selected embedding ψ and separator policy.

Source motif	Distilled mechanism	Correction or bound
Text holes and word breaks (lines 22–28)	Planar glyph mask with β_0, β_1, χ, plus a separate token count.	Font-free counts, four-torus language and mixing token count with glyph topology are rejected.
Leet strings (lines 30–37)	Explicit finite rewrite table.	There is no universal leet mapping; collisions and profile version are recorded.
"SDF to 19" (lines 39–44)	Semantic fiber membership or scalar residual $\lvert\nu(r) - 19\rvert$.	A semantic residual is not the spatial SDF of a rendered glyph.
ASCII grid (lines 62–96)	Left-aligned, NUL-extended, bytewise XOR witness.	It is not a generic bit shift and does not create bytes from unspecified memory.
Repeated masks (lines 86–95)	Swap lemma and XOR-parallelogram relation.	"Phase shift" is replaced with exact finite-vector algebra.
Topological hinge (lines 115–151)	Commuting semantic square plus selected profile invariants.	Equal β_1 alone cannot establish continuous deformability; the measured β_1 values do not match.
Cross-language tests (lines 174–268)	Cross-profile falsification method.	Two ad hoc failures do not prove a Dutch singularity or universal uniqueness.

2.2 Admission classes

Exact byte arithmetic is **RETAINED**. New typed maps are **ENGINEERING**. Useful statements kept after correction are **CORRECTED**. Alias, font and tokenizer-dependent results are **BOUNDED**. Four-torus, entropy-conservation, memory-creation and language-singularity conclusions are **REJECTED**.

3 The retained UGTS-KC 3.6 substrate

3.1 Referential world object

Version 3.6 defined

$$\mathcal{W}_{3.6} = (\mathcal{D}, \mathcal{X}, \mathcal{G}, \mathcal{R}, \mathcal{S}, \mathcal{C}, \mathcal{H}, \mathcal{E}, \mathcal{T}, \mathcal{I}, \mathcal{L}, \mathcal{P}),$$

where $\mathcal{D}$ is the literal definition registry, $\mathcal{X}$ instances, $\mathcal{G}$ finite grammar, $\mathcal{R}$ relations/fields, $\mathcal{S}$ support, $\mathcal{C}$ compatibility, $\mathcal{H}$ hinges, $\mathcal{E}$ events, $\mathcal{T}$ transitions, $\mathcal{I}$ invariants, $\mathcal{L}$ lineage/novelty log and $\mathcal{P}$ ordered pipelines.

Each definition remains content-addressed:

$$h(d) = \mathrm{SHA256}(\mathrm{canonicalJSON}(d \setminus \{h\})).$$

References must resolve and dependency graphs must be acyclic. The hash is verification metadata, not an ownership or authorship claim.

3.2 Query-first event order

The authoritative sequence remains:

$$\text{support} \to \text{compatibility} \to \text{guard root} \to \text{verified event} \to \text{transition} \to \text{lineage}.$$

Projection and rendering are consumers. One-bit fields remain narrow masks, route selectors or parity states, never complete geometry, identity, uncertainty or amplitude.

4 BEA architecture and formal object

4.1 Representation profile

FORMAL DEFINITION

A representation profile is

$$P = (\Sigma, N, \tau, \gamma, \epsilon, \nu, \Pi),$$

where Σ is the symbol/token domain, N normalization, τ an optional finite transducer, γ a glyph renderer, ϵ a byte encoder, ν a typed semantic evaluator and Π the versioned policy metadata.

No feature extracted under one profile is silently treated as profile-free. Font, case, spacing, Unicode normalization, tokenization, byte encoding and padding are part of the result.

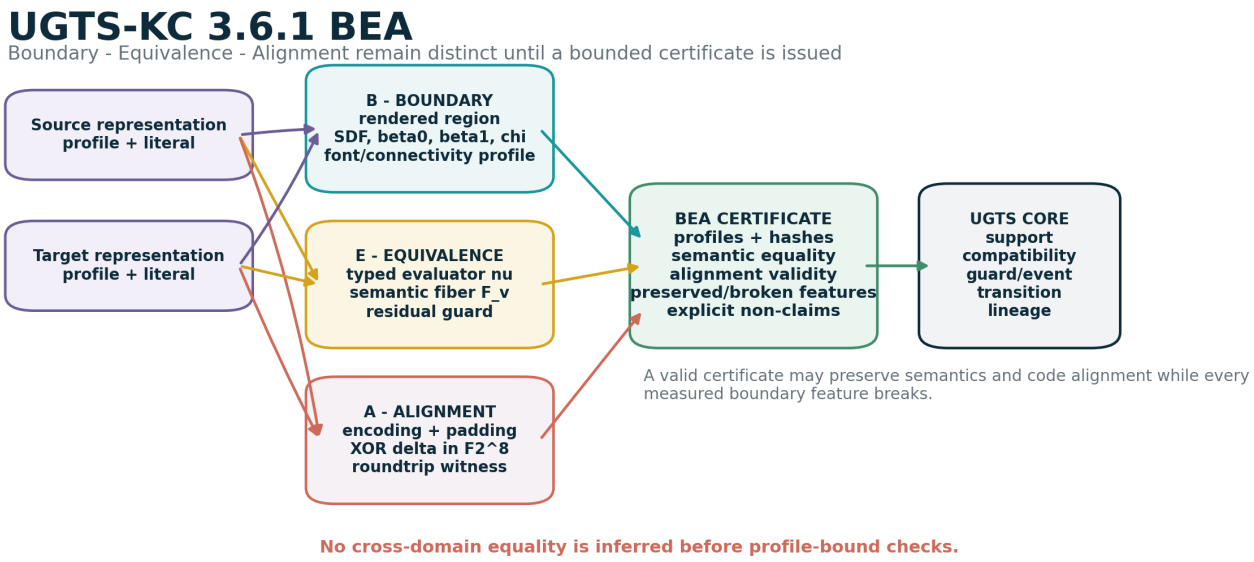


Figure 1: The three BEA spaces are evaluated separately before a bounded certificate enters the ordinary UGTS event pipeline.

4.2 Extended world

Version 3.6.1 extends the world with boundary profiles/signatures $\mathcal{B}$, alignment witnesses $\mathcal{A}$ and certificates $\mathcal{Q}$:

$$\mathcal{W}_{3.6.1} = (\mathcal{W}_{3.6}, \mathcal{B}, \mathcal{A}, \mathcal{Q}).$$

These are definition-backed records, not prose attached after execution.

4.3 BEA certificate

FORMAL DEFINITION

For source and target representations, a BEA certificate is

$$C_{BEA} = (r_s, r_t, P_s, P_t, B_s, B_t, E, A, I_+, I_-, N),$$

where B_s, B_t are optional boundary signatures, E the typed semantic comparison, A the exact alignment witness, I_+ selected verified equalities, I_- selected verified inequalities and N explicit non-claims.

A certificate may be valid even when every boundary feature differs. In that case it certifies only semantic equivalence under the chosen evaluators plus code alignment under the chosen encoding policy.

5 Boundary calculus

5.1 Rendered region and signed distance

For representation r under rendering profile P, let $G_P(r) \subset \mathbb{R}^2$ be the foreground region. Its spatial signed-distance field is

$$d_{G_P(r)}(x) = \begin{cases} -\operatorname{dist}(x, \partial G_P(r)), & x \in \operatorname{int} G_P(r), \\ 0, & x \in \partial G_P(r), \\ +\operatorname{dist}(x, \partial G_P(r)), & x \in \operatorname{ext} G_P(r). \end{cases}$$

Exact-distance status belongs to the renderer/geometry profile. A raster distance transform is a sampled approximation unless a vector distance method and tolerance are declared.

5.2 Digital planar topology

The shipped analyzer uses foreground 8-connectivity, background 4-connectivity and excludes the exterior background component from β_1. It returns

$$B_P(r) = (\beta_0, \beta_1, \chi, A_{px}, W, H), \qquad \chi = \beta_0 - \beta_1.$$

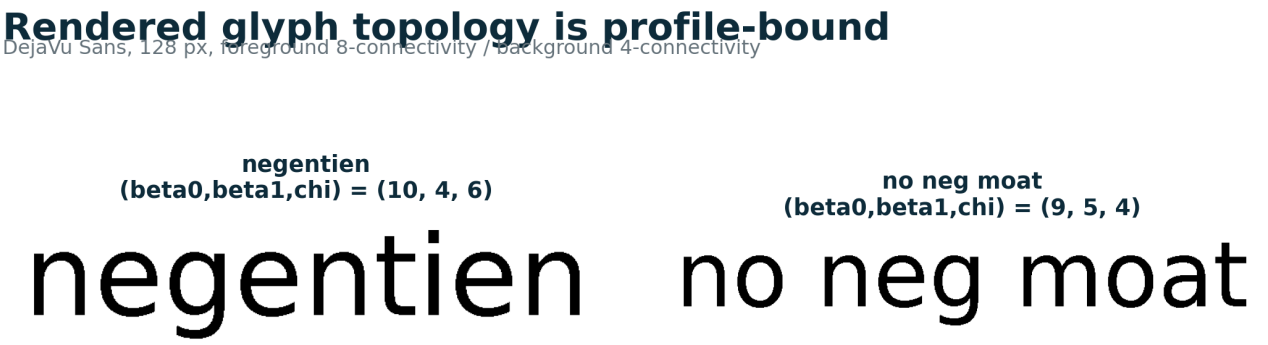
Detached dots and disconnected glyph parts contribute to β_0. Enclosed counters contribute to β_1. A different font or threshold may change both.

CORRECTION

The source states that *negentien* and *no neg moat* each have four holes and treats the first as one connected block and the second as three. This mixes three models. In ordinary text layout, separate glyphs are not joined into one foreground component; the dot of *i* is another component; token count is lexical, not glyph-topological; and the alias phrase also contains *e*. The package therefore measures glyph topology and token structure separately.

5.3 No torus inference

A planar set with k bounded complement components may have rank $H_1 = k$ in an appropriate model, but it is not a k-torus. A torus has a specific product/gluing construction, dimension and homology. Counting counters in letters does not supply that structure.



Font profile	negentien	no neg moat	beta1 equal?
DejaVuSans	(10,4,6)	(9,5,4)	NO
DejaVuSerif	(10,4,6)	(9,5,4)	NO
Lato-Regular	(9,5,4)	(9,6,3)	NO
InterDisplay-Regular	(10,4,6)	(9,5,4)	NO

The source's font-free beta1=4 equality is not reproduced. Token count (1 vs 3) is a different lexical model.

Figure 2: The source's font-free counter equality is not reproduced in four measured profiles.

5.4 Measured stress matrix

Font profile	Text	β_0	β_1	χ	pixels
DejaVuSans.ttf	negentien	10	4	6	17762
DejaVuSans.ttf	no neg moat	9	5	4	19901
DejaVuSerif.ttf	negentien	10	4	6	17012
DejaVuSerif.ttf	no neg moat	9	5	4	18823
Lato-Regular.ttf	negentien	9	5	4	15036
Lato-Regular.ttf	no neg moat	9	6	3	16745
InterDisplay-Regular.otf	negentien	10	4	6	15319
InterDisplay-Regular.otf	no neg moat	9	5	4	17016

The most stable observation is not a matching number. It is the failure of the proposed invariant under a modest profile change. This justifies making the profile a required part of the definition.

6 Equivalence calculus

6.1 Typed evaluator and semantic fiber

Let

$$\nu_P : R_P \rightharpoonup V$$

be a typed evaluator. Two profile/representation pairs are semantically equivalent when

$$(P_s, r_s) \sim_E (P_t, r_t) \iff \nu_{P_s}(r_s) = \nu_{P_t}(r_t).$$

The semantic fiber of value v is

$$F_v = \{(P, r) : \nu_P(r) = v\}.$$

The source-proposed alias is admitted only by an explicit evaluator profile with status *source-proposed; external linguistic validity not established*. The certificate verifies the declared evaluator result, not the linguistic truth of the alias.

6.2 Semantic residual is not a glyph SDF

For numeric $V \subseteq \mathbb{R}$, a useful guard is

$$g_v(P, r) = |\nu_P(r) - v|.$$

It reaches zero for members of F_v. This is a scalar residual in value space. It is not the signed distance from a spatial point to the rendered text boundary. A true distance from a representation to F_v requires a separately declared metric on representation space.

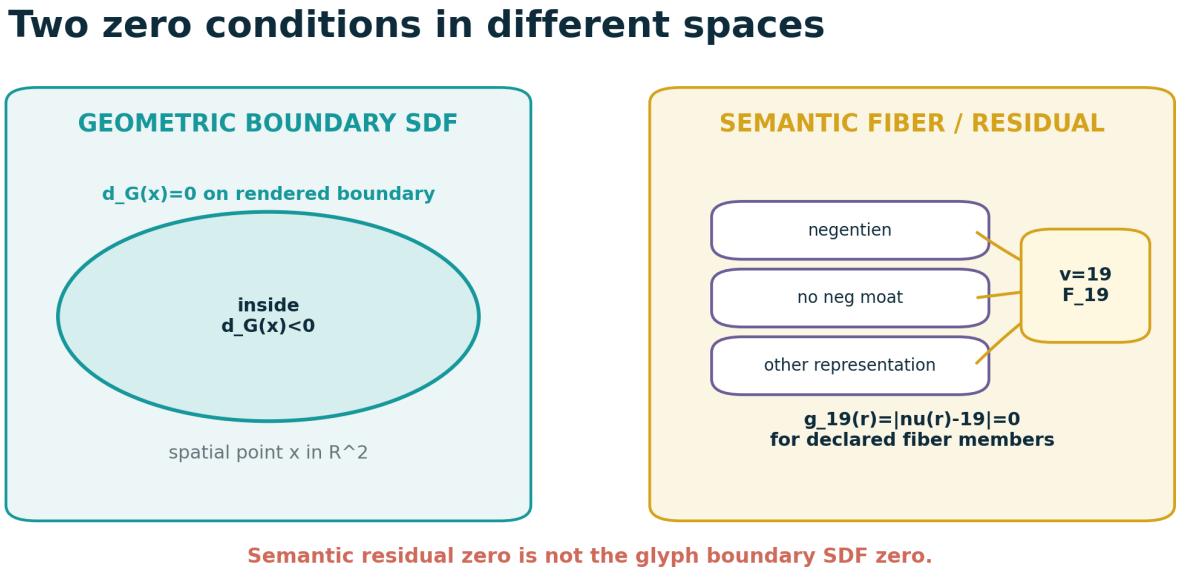


Figure 3: Glyph SDF zero and semantic residual zero are zero sets in different domains.

6.3 Commuting semantic hinge

A representation transform $H : R_s \to R_t$ is semantically preserving on a declared domain when

$$\nu_t \circ H = \nu_s.$$

This is the clean formal meaning of a representational hinge. It does not require the rendered regions to be homeomorphic.

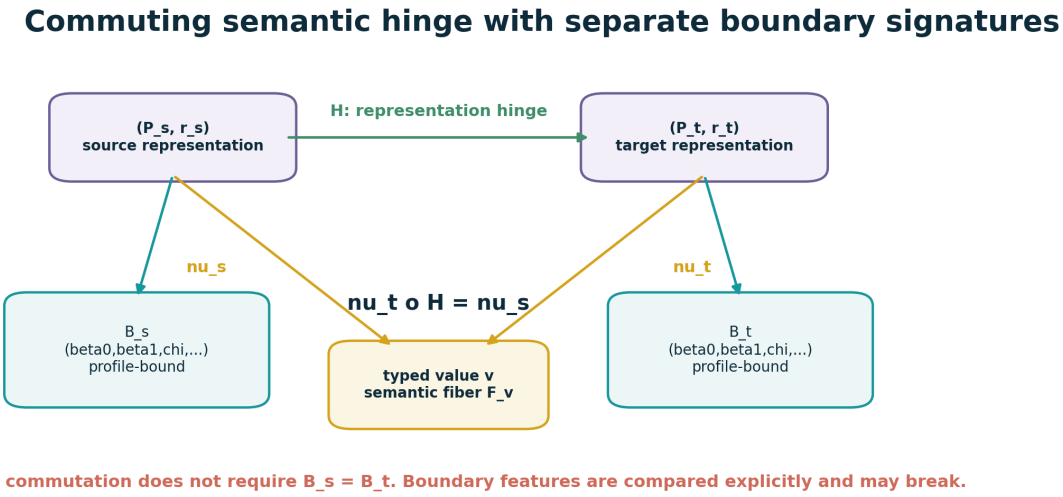


Figure 4: The semantic square may commute while boundary signatures differ.

7 Alignment calculus over binary vector spaces

7.1 Versioned leet transducer

The source uses the finite rewrite profile

$$a \mapsto 4, \qquad e \mapsto 3, \qquad i \mapsto 1, \qquad o \mapsto 0.$$

Under that profile:

$$\texttt{negentien} \mapsto \texttt{n3g3nt13n}, \qquad \texttt{no neg moat} \mapsto \texttt{n0 n3g m04t}.$$

This is a deterministic transducer, not a universal encoding.

7.2 Alignment and zero extension

Let e_s and e_t be encoded byte vectors. Choose common length L, alignment α and padding byte p:

$$\bar{e}_s = Z_{L,\alpha,p}(e_s), \qquad \bar{e}_t = Z_{L,\alpha,p}(e_t).$$

The source example uses ASCII, left alignment and $p = 0$. The shorter source is explicitly embedded into an 11-byte space. There is no operation on unspecified memory.

7.3 Exact XOR witness

Bytes are vectors in $\mathbb{F}_2^8$ and XOR is vector addition. Define

$$\Delta = \bar{e}_s \oplus \bar{e}_t.$$

Then

$$\bar{e}_s \oplus \Delta = \bar{e}_t.$$

This is an exact reversible witness under the declared alignment policy. Calling it a “bit shift” would be incorrect because bit positions inside bytes are not shifted left or right.

Exact ASCII XOR alignment witness

left-aligned, NUL-padded to 11 bytes; this is not a bit shift

i	src	source bits	mask	mask bits	target bits	dst
0	n	01101110	0x00	00000000	01101110	n
1	3	00110011	0x03	00000011	00110000	0
2	g	01100111	0x47	01000111	00100000	SP
3	3	00110011	0x5d	01011101	01101110	n
4	n	01101110	0x5d	01011101	00110011	3
5	t	01110100	0x13	00010011	01100111	g
6	1	00110001	0x11	00010001	00100000	SP
7	3	00110011	0x5e	01011110	01101101	m
8	n	01101110	0x5e	01011110	00110000	0
9	NUL	00000000	0x34	00110100	00110100	4
10	NUL	00000000	0x74	01110100	01110100	t

0x5d at indices 3,4: swap lemma (3,n)->(n,3)
0x5e at indices 7,8: XOR parallelogram relation

bit weights: source 39, target 37, delta 38
parity preserved (1 -> 1), but Hamming weight is not

Figure 5: The complete source-to-target XOR witness, with two repeated-mask classes.

7.4 Full byte table

i	src	source bits	mask	mask bits	target bits	dst
0	n	01101110	0x00	00000000	01101110	n
1	3	00110011	0x03	00000011	00110000	0
2	g	01100111	0x47	01000111	00100000	SP
3	3	00110011	0x5d	01011101	01101110	n
4	n	01101110	0x5d	01011101	00110011	3
5	t	01110100	0x13	00010011	01100111	g
6	1	00110001	0x11	00010001	00100000	SP
7	3	00110011	0x5e	01011110	01101101	m
8	n	01101110	0x5e	01011110	00110000	0
9	NUL	00000000	0x34	00110100	00110100	4
10	NUL	00000000	0x74	01110100	01110100	t

7.5 Swap lemma

For an aligned swap $(a, b) \mapsto (b, a)$,

$$(a \oplus b,\ b \oplus a) = (m, m).$$

This explains mask `0x5d` at indices 3 and 4: the pair `3n` becomes `n3`.

7.6 XOR parallelogram lemma

Two substitutions $a \mapsto c$ and $b \mapsto d$ share a mask exactly when

$$a \oplus c = b \oplus d \iff a \oplus b \oplus c \oplus d = 0.$$

This is an affine parallelogram relation in $\mathbb{F}_2^8$. It explains `0x5e` at indices 7 and 8:

$$\texttt{0x33} \oplus \texttt{0x6d} = \texttt{0x6e} \oplus \texttt{0x30} = \texttt{0x5e}.$$

7.7 Parity law and non-conservation of weight

Let $w(x)$ be Hamming weight and $\pi(x) = w(x) \bmod 2$. Because parity is linear,

$$\pi(\bar{e}_t) = \pi(\bar{e}_s) \oplus \pi(\Delta).$$

The measured values are

$$w(\bar{e}_s) = 39, \qquad w(\bar{e}_t) = 37, \qquad w(\Delta) = 38.$$

Thus Hamming weight is not conserved. Since 38 is even, parity is preserved: $1 \mapsto 1$. No entropy equality follows from this parity fact.

VERIFIED RESULT

The exact algebraic result is stronger and narrower than the source rhetoric: the alignment is reversible; two repeated-mask identities are explained; parity is preserved for this witness; Hamming weight is not preserved.

8 Partial invariants and the BEA claim ladder

8.1 Selected features

Let a signature map be

$$\sigma_P(r) = (\nu_P(r), T(r), \beta_0, \beta_1, \chi, |e|, w(e), \pi(e), \ldots).$$

A certificate names a projection $\pi_I \sigma$ that it expects to preserve. It must also record selected coordinates that break. This prevents one matching number from being mistaken for total equivalence.

8.2 Claim levels

1. **Typed semantic equivalence**: both evaluators return the same value.
2. **Exact code alignment**: a reversible witness connects encoded vectors under a profile.

3. **Profile feature match**: selected boundary or sequence features agree under declared profiles.
4. **Topological equivalence**: requires an explicit map and stronger evidence; not established here.

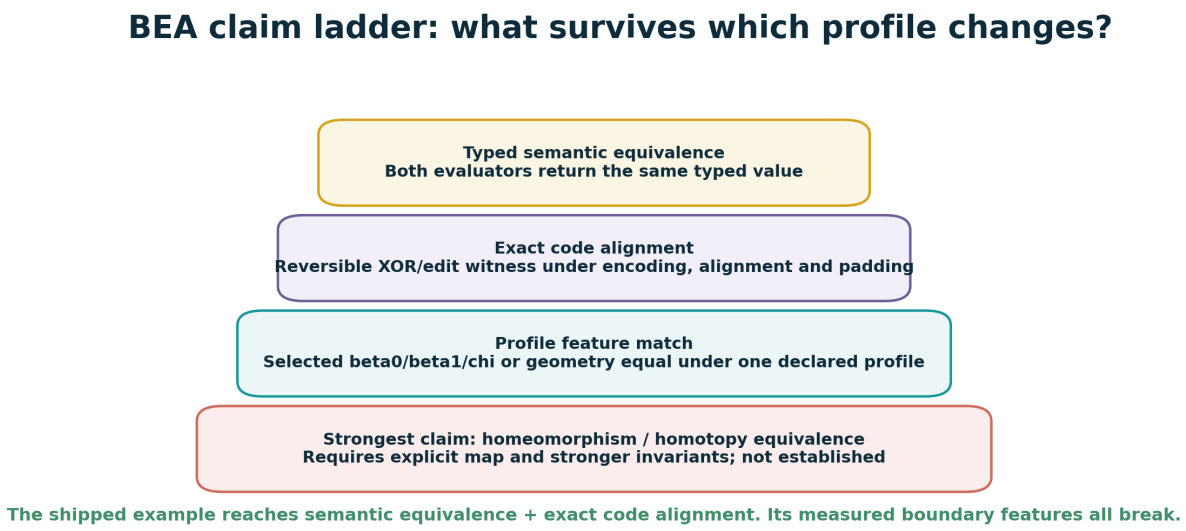


Figure 6: The shipped pair reaches the two lower levels and explicitly stops there.

8.3 Shipped certificate result

The literal example records:

- semantic values 19 and 19: equal under the declared evaluators;
- leet strings `n3g3nt13n` and `n0 n3g m04t`;
- valid 11-byte XOR roundtrip;
- source/target/delta weights $39/37/38$;
- parity preserved;
- boundary comparison under DejaVu Sans: all of β_0, β_1, χ broken;
- claim level `semantic-equivalence-plus-code-alignment`;
- explicit denial of linguistic proof, homotopy/homeomorphism and physical conservation.

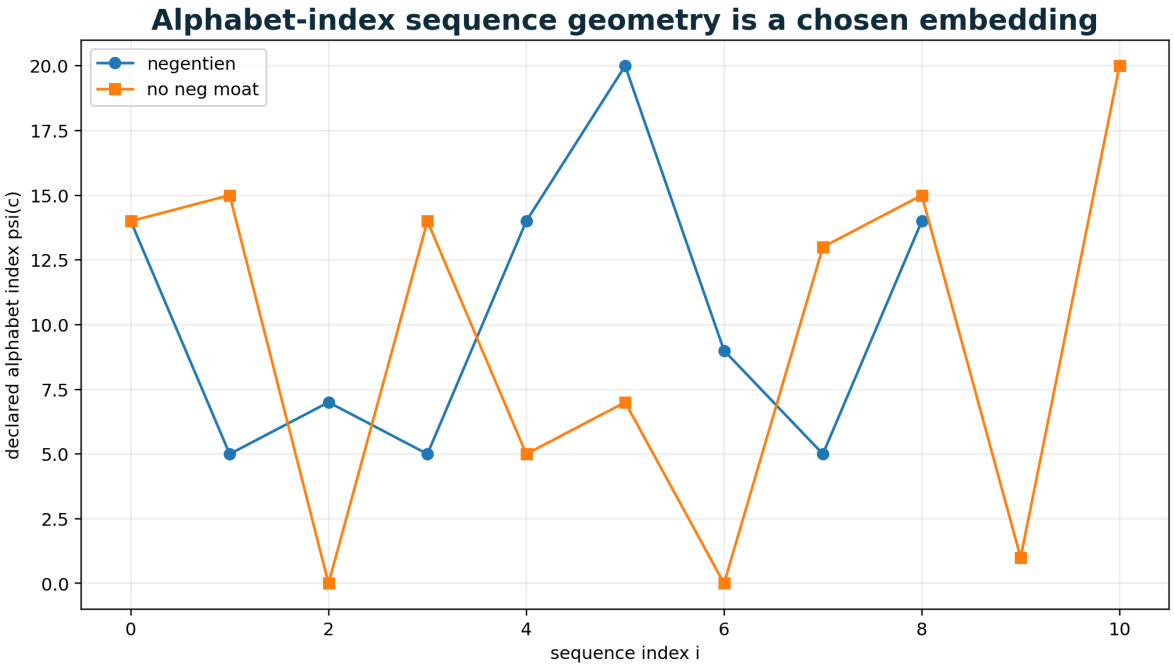
9 Sequence geometry and stress testing

9.1 Alphabet-index path

A declared alphabet embedding $\psi : \Sigma \to \mathbb{R}$ yields

$$p_i = (i, \psi(c_i)).$$

This makes the source's "spatial distribution" idea measurable: bounding box, centroid, variance, path length, repetition and autocorrelation may be computed. The result remains profile-bound because a different alphabet, normalization or separator value changes the path.



Bounding boxes and variances are measurable, but they depend on alphabet, normalization and separator policy.

Figure 7: A chosen symbol embedding creates analyzable sequence geometry without claiming intrinsic coordinates.

9.2 Falsification matrix

A candidate invariant should be stress-tested over:

- font family, glyph variant, size, stroke/fill, spacing and threshold;
- Unicode normalization, case and tokenization;
- byte encoding, alignment and padding;
- leet/transduction profile;
- language or dialect evaluator;
- digital foreground/background connectivity.

One counterexample limits the claim. Repeated success over a finite sample still does not prove universality. The source's French/German exercise is retained only as the idea of out-of-profile testing; its uniqueness conclusion is not carried forward.

10 Literal referential integration

10.1 New definition kinds

The example adds literal definitions for text representation, leet transduction, semantic pair comparison, XOR alignment, boundary signature comparison, BEA certificate and a 13-phase evaluation schedule. Every record has a stable ID, typed domain/codomain, dependencies, provenance, capabilities, invariants/non-invariants and canonical SHA-256 content hash.

10.2 Normative operation order

The BEA schedule is:

1. parse literals and schemas;
2. normalize Unicode, case, whitespace and units;
3. resolve references and verify hashes;

4. apply declared finite transducers;
5. evaluate typed semantics;
6. encode, align and construct code witnesses;
7. render/extract boundary signatures;
8. compare selected preserved and broken features;
9. issue the BEA certificate;
10. apply ordinary local support;
11. apply compatibility;
12. solve/verify relation guards and transitions;
13. append lineage and irreducible novelty.

The order is non-commutative. In particular, transduction before byte encoding differs from byte encoding before text rewriting, and semantic comparison cannot be inferred from boundary comparison.

10.3 Literal example excerpt


```
{
  "$schema": "../spec/ugts_kc_3_6_1_bea.schema.json",
  "schema_version": "3.6.1",
  "substrate_id": "substrate:ugts-kc-3.6.1-bea-example",
  "metadata": {
    "title": "UGTS-KC 3.6.1 BEA literal referential example",
    "version_title": "BEA",
    "bea_expansion": "Boundary-Equivalence-Alignment",
    "description": "Carries the 3.6 number-to-geometry pipeline and a pairwise BEA certificate pipeline.",
    "evidence_boundary": "The phrase-to-value alias is source-proposed and not independently validated as Dutch phonetics. Glyph topology is profile-bound. XOR is exact only under the declared ASCII, left-alignment and NUL-padding policy.",
    "requester_attribution": {
      "name": "Tom Klootwijk",
      "identifier": "NL200678942",
      "date_of_birth": "10-07-1990",
      "status": "requester-supplied-unverified"
    }
  },
  "definitions": [
    {
      "id": "def:number-literal-v1",
      "kind": "literal_integer",
      "domain": "{}",
      "codomain": "integer[0,99]",
      "evaluation_phase": 0,
      "dependencies": [],
      "parameters": {},
      "provenance": {
        "class": "engineering-derived",
        "note": "Literal numeric instance family.",
        "source_refs": []
      },
      "content_hash": "sha256:79ad07905ca506b17b0f5947948a25495bbe3a2386e110b6e4b65226c6f39e16"
    },
    {
      "id": "op:binary-radix-v1",
      "kind": "radix_encode",
      "domain": "integer[0,99]",
      "codomain": "radix_word",
      "evaluation_phase": 1,
      "dependencies": [],
      "parameters": {
        "base": 2
      },
      "provenance": {
        "class": "source-derived",
        "note": "Exact positional encoding.",
        "source_refs": [
          "SRC-MD lines 29-40",
          "UGTS-GN M001-M004"
        ]
      },
      "content_hash": "sha256:35d697e690e3e3eb2f5f8ebd7c343ad73e1cb4c1b95ccd9d76507bbef47bcc50"
    },
    {
      "id": "op:active-bits-v1",
      "kind": "active_bit_filter",
      "domain": "integer[0,99]",
      "codomain": "active_bit_set",
      "evaluation_phase": 2,
      "dependencies": [
        "op:binary-radix-v1"
      ],
```

```
      "parameters": {},
      "provenance": {
        "class": "source-derived",
        "note": "Set-bit extraction without changing numeric identity.",
        "source_refs": [
          "SRC-MD lines 130-142",
          "UGTS-GN M006-M008"
        ]
      },
      "content_hash": "sha256:536b3c4408df71152ded0a91e2e290d349154808b1642dde8cb1823979681498"
```


10.4 Reference runtime

The dependency-free core exposes:

- `LeetProfile.encode`;
- `alphabet_index_path`;
- `xor_align`, `xor_same_mask`, `bit_weight`, `bit_parity`;
- `BoundarySignature`, `compare_boundary_signatures`;
- `semantic_equivalent`, `numeric_semantic_residual`;
- `build_bea_certificate`;
- `Runtime.execute_pair` for literal pair pipelines.

The optional analysis tool renders masks using Pillow and measures components/holes using complementary digital connectivity. Font files are referenced locally but never redistributed.

11 Kinematic and event-calculus use

11.1 Representation transitions as guarded hinges

A time-dependent representational map $H_{\lambda(t)}$ may interpolate geometry or switch profiles. The continuous geometry may be queried at arbitrary t, while a certificate/state transition is committed only when a declared guard crosses and support/compatibility hold.

For a scalar relation $R(q(t)) = 0$, the ordinary UGTS event remains

$$t^* = \min\{t > t_0 : R(q(t)) = 0,\ S(q,t) = 1,\ \chi(q,t) = 1,\ conf \geq conf_{min}\}.$$

BEA does not replace that event calculus. It supplies a typed precondition or evidence record for representation-changing transitions.

11.2 Example event policy

A system may require:

1. semantic square commutes;
2. alignment witness verifies;
3. selected boundary invariants hold within tolerance;
4. no forbidden non-invariant changes occur;
5. only then may a route or representation state change be committed.

A different application may allow boundary features to break while preserving semantic identity. The policy is explicit rather than hidden in a metaphorical “hinge.”

12 Reusable BEA pattern recipes

ID	Recipe	Input / pipeline	Boundary
R361-01	Profile-bound glyph topology	text + font/render profile: render mask -> label foreground/background -> beta0,beta1,chi	Never report counters without the profile.
R361-02	Token/glyph separation	text: tokenize -> token_count; independently render -> glyph beta0	Do not identify word count with glyph connected components.
R361-03	Semantic fiber bridge	representations + evaluators: evaluate both -> compare typed values -> common fiber key	Establishes semantic equality only.
R361-04	Semantic residual guard	representation + target value: evaluate -> absolute residual -> threshold/event	Call it a guard unless a representation-space metric is defined.
R361-05	Leet rewrite profile	text + substitution table: normalize -> finite rewrite -> preserve profile id	Mappings are versioned and may collide.
R361-06	Padded XOR witness	two byte strings: encode -> align -> pad -> XOR -> verify roundtrip	Padding and original lengths are part of the result.
R361-07	Swap symmetry detector	aligned byte pairs: detect (a,b)->(b,a) -> duplicate mask	A local algebraic symmetry, not physical phase.
R361-08	XOR parallelogram detector	two aligned substitutions: test a xor b xor c xor d == 0	Groups equal-mask substitutions in F2^8.
R361-09	Parity audit	source,target,delta: compute bit weights and parity -> verify linear parity law	Parity preservation does not preserve weight or entropy.
R361-10	Boundary/equivalence separation	text pair: compute glyph SDF/signatures separately from semantic evaluator	Prevents semantic zero from masquerading as geometric SDF zero.
R361-11	Partial hinge certificate	source,target,selected features: verify semantic square + alignment + selected equalities + non-claims	Does not imply full topology equivalence.
R361-12	Font stress matrix	text pair + fonts: repeat topology extraction per font -> tabulate changes	A single counterexample bounds font-independent claims.
R361-13	Alignment stress matrix	text pair + policies: repeat left/right and encodings -> compare witnesses	Code geometry is policy-dependent.
R361-14	Cross-profile falsification	candidate invariant + profiles: evaluate all profiles -> retain/fail with counterexample	Failure limits scope; success over a sample is not universality.
R361-15	Symbol path geometry	text + symbol embedding: map to sequence path -> bbox, variance, length, autocorrelation	Coordinates come from the chosen embedding.
R361-16	BEA event pipeline	literal definitions: normalize -> B/E/A -> certificate -> support -> compatibility -> guard -> lineage	Certificate issuance is ordered and auditable.

13 Validation and reproducibility

13.1 Automated tests

The package runs the 36 baseline tests and 30 BEA tests, for 66 total. They cover canonical hashing, literal definition resolution, Dutch number profiles, geometry, topology, quotient maps, XOR alignment, repeated-mask lemmas, parity, boundary signature contracts, semantic typing, certificate validation, JSON Schema and pairwise runtime traces.

```
test_01_canonical_key_order (test_ugts36.CanonicalTests.test_01_canonical_key_order) ... ok
test_02_hash_is_stable (test_ugts36.CanonicalTests.test_02_hash_is_stable) ... ok
test_03_hash_changes_with_semantics (test_ugts36.CanonicalTests.test_03_hash_changes_with_semantics) ... ok
test_04_attach_and_verify (test_ugts36.CanonicalTests.test_04_attach_and_verify) ... ok
test_13_negentien_profile (test_ugts36.DutchPhoneticTests.test_13_negentien_profile) ... ok
test_14_drieentwintig_profile (test_ugts36.DutchPhoneticTests.test_14_drieentwintig_profile) ... ok
test_15_place_and_spoken_order_are_distinct (test_ugts36.DutchPhoneticTests.test_15_place_and_spoken_order_are_distinct) ... ok
test_16_multiple_of_ten_has_no_connector (test_ugts36.DutchPhoneticTests.test_16_multiple_of_ten_has_no_connector) ... ok
test_17_zeventien_segments (test_ugts36.DutchPhoneticTests.test_17_zeventien_segments) ... ok
test_18_elf_is_irregular (test_ugts36.DutchPhoneticTests.test_18_elf_is_irregular) ... ok
test_19_ninety_nine_profile (test_ugts36.DutchPhoneticTests.test_19_ninety_nine_profile) ... ok
test_20_lexicon_is_complete_and_unique (test_ugts36.DutchPhoneticTests.test_20_lexicon_is_complete_and_unique) ... ok
test_21_pulse_match_set_is_declared (test_ugts36.DutchPhoneticTests.test_21_pulse_match_set_is_declared) ... ok
test_22_profile_bounds (test_ugts36.DutchPhoneticTests.test_22_profile_bounds) ... ok
test_05_digit_count (test_ugts36.GeometryTests.test_05_digit_count) ... ok
test_06_radix_digits (test_ugts36.GeometryTests.test_06_radix_digits) ... ok
test_07_active_bits (test_ugts36.GeometryTests.test_07_active_bits) ... ok
test_08_regular_triangle_has_area (test_ugts36.GeometryTests.test_08_regular_triangle_has_area) ... ok
test_09_point_and_segment_embeddings (test_ugts36.GeometryTests.test_09_point_and_segment_embeddings) ... ok
test_10_rotation_preserves_pivot (test_ugts36.GeometryTests.test_10_rotation_preserves_pivot) ... ok
test_11_logpolar_roundtrip (test_ugts36.GeometryTests.test_11_logpolar_roundtrip) ... ok
test_12_sdf_and_csg_signs (test_ugts36.GeometryTests.test_12_sdf_and_csg_signs) ... ok
test_31_json_schema_validation (test_ugts36.SubstrateRuntimeTests.test_31_json_schema_validation) ... ok
test_32_substrate_load_and_order (test_ugts36.SubstrateRuntimeTests.test_32_substrate_load_and_order) ... ok
test_33_all_definition_hashes_verify (test_ugts36.SubstrateRuntimeTests.test_33_all_definition_hashes_verify) ... ok
test_34_runtime_trace_for_19 (test_ugts36.SubstrateRuntimeTests.test_34_runtime_trace_for_19) ... ok
test_35_runtime_trace_for_23 (test_ugts36.SubstrateRuntimeTests.test_35_runtime_trace_for_23) ... ok
test_36_missing_reference_is_rejected (test_ugts36.SubstrateRuntimeTests.test_36_missing_reference_is_rejected) ... ok
test_23_permutation_connector_hinge (test_ugts36.TopologyHingeTests.test_23_permutation_connector_hinge) ... ok
test_24_bad_permutation_is_rejected (test_ugts36.TopologyHingeTests.test_24_bad_permutation_is_rejected) ... ok
test_25_affine_composition_is_noncommutative (test_ugts36.TopologyHingeTests.test_25_affine_composition_is_noncommutative) ... ok
test_26_mobius_flip (test_ugts36.TopologyHingeTests.test_26_mobius_flip) ... ok
test_27_klein_flip_and_sheet_toggle (test_ugts36.TopologyHingeTests.test_27_klein_flip_and_sheet_toggle) ... ok
test_28_linear_crossing_time (test_ugts36.TopologyHingeTests.test_28_linear_crossing_time) ... ok
test_29_hourglass_routes (test_ugts36.TopologyHingeTests.test_29_hourglass_routes) ... ok
test_30_topological_order_and_cycle (test_ugts36.TopologyHingeTests.test_30_topological_order_and_cycle) ... ok
test_16_planar_boundary_signature (test_ugts361_bea.BoundaryAndSemanticTests.test_16_planar_boundary_signature) ... ok
test_17_invalid_euler_characteristic_rejected
↪  (test_ugts361_bea.BoundaryAndSemanticTests.test_17_invalid_euler_characteristic_rejected) ... ok
test_18_all_measured_features_break (test_ugts361_bea.BoundaryAndSemanticTests.test_18_all_measured_features_break) ... ok
test_19_source_hole_match_does_not_hold_in_measured_profile
↪  (test_ugts361_bea.BoundaryAndSemanticTests.test_19_source_hole_match_does_not_hold_in_measured_profile) ... ok
test_20_token_components_are_not_glyph_components
↪  (test_ugts361_bea.BoundaryAndSemanticTests.test_20_token_components_are_not_glyph_components) ... ok
test_21_semantic_equality_is_typed (test_ugts361_bea.BoundaryAndSemanticTests.test_21_semantic_equality_is_typed) ... ok
test_22_semantic_residual_is_not_signed (test_ugts361_bea.BoundaryAndSemanticTests.test_22_semantic_residual_is_not_signed) ... ok
test_23_certificate_without_boundary (test_ugts361_bea.CertificateTests.test_23_certificate_without_boundary) ... ok
test_24_broken_boundary_features_do_not_invalidate_bounded_certificate
↪  (test_ugts361_bea.CertificateTests.test_24_broken_boundary_features_do_not_invalidate_bounded_certificate) ... ok
test_25_false_preserved_feature_claim_invalidates_certificate
↪  (test_ugts361_bea.CertificateTests.test_25_false_preserved_feature_claim_invalidates_certificate) ... ok
test_01_source_leet_profile (test_ugts361_bea.LeetAndPathTests.test_01_source_leet_profile) ... ok
test_02_profile_is_explicit (test_ugts361_bea.LeetAndPathTests.test_02_profile_is_explicit) ... ok
test_03_symbol_path_is_sequence_geometry (test_ugts361_bea.LeetAndPathTests.test_03_symbol_path_is_sequence_geometry) ... ok
test_04_separator_policy_is_profile_bound (test_ugts361_bea.LeetAndPathTests.test_04_separator_policy_is_profile_bound) ... ok
test_26_schema_validation (test_ugts361_bea.SchemaAndRuntimeTests.test_26_schema_validation) ... ok
test_27_substrate_loads_and_hashes_verify (test_ugts361_bea.SchemaAndRuntimeTests.test_27_substrate_loads_and_hashes_verify) ... ok
test_28_runtime_pair_transduction_and_alignment
↪  (test_ugts361_bea.SchemaAndRuntimeTests.test_28_runtime_pair_transduction_and_alignment) ... ok
test_29_runtime_boundary_comparison_is_profile_bound
↪  (test_ugts361_bea.SchemaAndRuntimeTests.test_29_runtime_boundary_comparison_is_profile_bound) ... ok
test_30_runtime_certificate_is_bounded (test_ugts361_bea.SchemaAndRuntimeTests.test_30_runtime_certificate_is_bounded) ... ok
test_05_bit_weights_match_reproduction (test_ugts361_bea.XorAlignmentTests.test_05_bit_weights_match_reproduction) ... ok
test_06_left_alignment_lengths (test_ugts361_bea.XorAlignmentTests.test_06_left_alignment_lengths) ... ok
test_07_xor_witness_roundtrip (test_ugts361_bea.XorAlignmentTests.test_07_xor_witness_roundtrip) ... ok
test_08_repeated_mask_classes (test_ugts361_bea.XorAlignmentTests.test_08_repeated_mask_classes) ... ok
test_09_hamming_weight_is_not_preserved (test_ugts361_bea.XorAlignmentTests.test_09_hamming_weight_is_not_preserved) ... ok
test_10_parity_is_preserved_for_this_witness (test_ugts361_bea.XorAlignmentTests.test_10_parity_is_preserved_for_this_witness) ...
↪  ok
test_11_right_alignment_is_a_different_contract
↪  (test_ugts361_bea.XorAlignmentTests.test_11_right_alignment_is_a_different_contract) ... ok
test_12_bad_alignment_rejected (test_ugts361_bea.XorAlignmentTests.test_12_bad_alignment_rejected) ... ok
test_13_swap_pair_has_duplicate_xor_mask (test_ugts361_bea.XorAlignmentTests.test_13_swap_pair_has_duplicate_xor_mask) ... ok
test_14_affine_xor_parallelogram (test_ugts361_bea.XorAlignmentTests.test_14_affine_xor_parallelogram) ... ok
test_15_non_parallelogram (test_ugts361_bea.XorAlignmentTests.test_15_non_parallelogram) ... ok

----------------------------------------------------------------------
Ran 66 tests in 0.012s

OK
```

13.2 What the tests establish

They establish internal consistency and reproducibility of the bounded package. They do not establish the source-proposed alias as standard Dutch, validate every font or language, prove topological equivalence or benchmark physical hardware.

13.3 Reproduction commands

```
cd UGTS_KC_3_6_1_BEA_Tom_Klootwijk
PYTHONPATH=src python -m unittest discover -s tests -v
PYTHONPATH=src python examples/demo_bea.py
python tools/analyze_glyph_topology.py --out-dir data/bea_analysis
python tools/generate_bea_figures.py
```

14 Claims ledger

ID	Claim	Disposition	Technical treatment
C361-01	The source-proposed phrase is standard Dutch phonetics.	BOUNDED/ UNVERIFIED	Store as an explicit alias profile; the supplied document does not establish dialect evidence.
C361-02	Alphabet positions reveal intrinsic word geometry.	BOUNDED	They define one chosen embedding whose statistics can be measured.
C361-03	Both rendered strings have four holes independent of font.	REJECT	Measured profiles in the package give beta1 4 vs 5 in DejaVu Sans and 5 vs 6 in Lato.
C361-04	Four holes make a four-torus.	REJECT	A planar region with beta1=4 is not a four-dimensional/product torus.
C361-05	One block versus three words trades beta0 for beta1.	REJECT	Token topology and glyph-mask topology are different models; no conservation law is shown.
C361-06	The ASCII table is a bit-shift.	CORRECTED	It is a padded, left-aligned bytewise XOR delta.
C361-07	Repeated masks are random-free phase shifts.	CORRECTED	0x5d is explained by a swap; 0x5e by an XOR parallelogram relation.
C361-08	XOR creates final bytes from empty memory.	REJECT	The shorter vector is explicitly zero-extended before XOR.
C361-09	High-bit/low-bit ratio is preserved.	REJECT	Source and target Hamming weights are 39 and 37.
C361-10	Global parity is preserved in the shipped example.	SUPPORTED/ BOUNDED	Both parities are 1 because the mask has even weight 38.
C361-11	Equal semantic value means geometric SDF zero.	CORRECTED	It gives zero semantic residual or fiber distance under a declared representation metric, not the glyph SDF.
C361-12	Matching beta1 proves continuous deformation.	REJECT	Full homotopy/homeomorphism needs a stronger map and more invariants; in measured profiles beta1 does not match anyway.
C361-13	French/German failures prove a Dutch singularity.	REJECT	The proposed aliases and glyph profiles are ad hoc; failures only show dependence on those profiles.
C361-14	A BEA certificate binds representations safely.	SUPPORTED WITH BOUNDS	It binds declared semantic value, exact code alignment and selected profile features while recording non-claims.

15 Acceptance and kill criteria

A BEA use should be rejected or narrowed when:

- the representation profile omits font, normalization, encoding or alignment policy;
- semantic evaluators are ambiguous, circular or unvalidated but treated as facts;
- token count is mixed with rendered β_0;
- a counter count is reported without font/connectivity/threshold metadata;
- equality of one Betti number is promoted to homeomorphism or continuous deformation;
- semantic residual is described as the spatial glyph SDF;

- an XOR witness is called a shift or an entropy invariant;
- padding and original lengths are not retained;
- a repeated mask is treated as causal or physical symmetry;
- profile failures are rebranded as proof of uniqueness;
- certificate cost or complexity exceeds the materialization or verification it replaces.

16 Conclusion

The attached document contains a useful structural idea beneath incorrect escalations: different representations may be connected through explicit, testable maps. Version 3.6.1 makes that idea precise. Boundary geometry/topology, semantic value and code alignment are separate spaces. Their results meet only in a typed certificate that states exactly what is preserved, what changes and what is not claimed.

The strongest exact trick in the source is the XOR table. After explicit NUL extension it is a reversible alignment witness. Its duplicate masks reveal a swap identity and an affine parallelogram in $\mathbb{F}_2^8$. The strongest corrective result is the glyph-topology stress test: the proposed equal counter count does not survive ordinary font profiles, and token count is a separate model. The strongest new formal object is the commuting BEA hinge, which can preserve semantic value while boundary geometry changes.

DECISION

UGTS-KC 3.6.1 BEA is ready as a bounded referential extension. It strengthens the formal definition by refusing to merge domains: geometry remains geometry, topology remains profile-bound and explicit, semantics remain evaluator-bound, binary alignment remains exact code algebra, and all cross-domain claims require a certificate with a non-claim ledger.

A Complete 3.6.1 BEA operator catalog

The 32 operators below are the 3.6.1 delta namespace. The 36 version-3.6 operators and the earlier normalized baseline remain unchanged in the package.

ID	Domain	Operator	Definition / disposition / validation
BEA-01	Referential representation	Representation profile	A representation profile declares alphabet/tokenizer, normalization, semantic evaluator, glyph renderer, byte encoder and version. [ENGINEERING; Schema + tests]
BEA-02	Referential representation	Typed normalization	Unicode normalization, case policy, whitespace policy and tokenization occur before any geometric, semantic or byte comparison. [RETAIN; Implemented subset]
BEA-03	Phonetic/transduction	Source-proposed alias transducer	A proposed phonetic alias is stored as a versioned relation with external-validity status; it is not promoted to a linguistic theorem. [CORRECTED; Schema boundary]
BEA-04	Encoding	Versioned leet transducer	A finite character rewrite table such as a->4, e->3, i->1, o->0 is explicit and non-universal. [BOUNDED; Implemented]
BEA-05	Sequence geometry	Alphabet-index path	A declared symbol embedding maps character sequence i to points (i,psi(c_i)); bounding box, centroid, variance and path length are measurable profile features. [ENGINEERING; Implemented]
BEA-06	Lexical topology	Token component signature	Whitespace-separated token count is lexical connectivity metadata and must not be confused with connected components of rendered glyphs. [CORRECTED; Implemented]
BEA-07	Boundary geometry	Glyph rendering profile	Font, size, style, spacing, antialias threshold, fill/stroke and connectivity are mandatory before glyph topology is measured. [RETAIN; Measured profiles]
BEA-08	Boundary topology	Planar mask complex	A rendered foreground region is treated as a planar digital cell complex under declared foreground/background connectivity. [ENGINEERING; Analyzer]
BEA-09	Boundary topology	Complementary connectivity contract	Foreground 8-connectivity and background 4-connectivity, or the reverse, prevent ambiguous digital adjacency; the exterior background is excluded from hole count. [RETAIN; Analyzer]
BEA-10	Boundary topology	Beta-zero component count	beta0 counts connected foreground components in the declared rendered region; detached dots and glyph parts count unless the profile bridges them. [RETAIN; Measured]
BEA-11	Boundary topology	Beta-one counter count	beta1 counts bounded background components/counters of the declared planar region; it is font- and rendering-dependent. [CORRECTED; Measured]
BEA-12	Boundary topology	Planar Euler signature	For the shipped planar model, chi=beta0-beta1. Equality of beta1 alone is not topological equivalence. [RETAIN; Tested]
BEA-13	Boundary topology	No torus inference	A planar text mask with k counters is not a k-torus; dimension, homology groups and gluing structure differ. [CORRECTED; Claims ledger]
BEA-14	Boundary geometry	Glyph signed-distance field	d_G(x) is the signed Euclidean distance from a spatial point to the rendered glyph boundary, negative inside and positive outside. [RETAIN; Formal specification]
BEA-15	Semantic equivalence	Typed evaluator	Each representation is interpreted by a declared evaluator nu_P into a typed value space. [ENGINEERING; Runtime subset]
BEA-16	Semantic equivalence	Semantic fiber	F_v is the set of profile/representation pairs evaluating to v; membership establishes semantic equivalence only under those evaluators. [ENGINEERING; Formal definition]
BEA-17	Semantic equivalence	Semantic residual guard	For numeric values, |nu(r)-v| is a scalar residual/guard. It is not automatically a geometric SDF on text shapes. [CORRECTED; Implemented]
BEA-18	Alignment	Explicit alignment policy	Encoding, left/right alignment, original lengths and padding byte are part of the witness. [RETAIN; Implemented]
BEA-19	Alignment	Zero-extension embedding	Shorter byte vectors are embedded into a common vector space by explicit padding; XOR does not materialize bytes from unspecified memory. [CORRECTED; Implemented]
BEA-20	Alignment	Byte vector over F2^8	Each byte is a vector in the eight-dimensional binary vector space; XOR is vector addition. [RETAIN; Formal + tests]
BEA-21	Alignment	Reversible XOR delta	Delta=E_s xor E_t is an exact witness because E_s xor Delta=E_t under the declared alignment. [RETAIN; Implemented]
BEA-22	Alignment	Repeated-mask class	Indices with the same XOR byte form an observed equivalence class for that exact aligned pair, not a universal phase symmetry. [CORRECTED; Implemented]
BEA-23	Alignment	Swap-pair lemma	An aligned swap (a,b)->(b,a) yields the duplicated mask (a xor b,a xor b). [RETAIN; Tested]
BEA-24	Alignment	XOR parallelogram lemma	Two substitutions a->c and b->d share a mask iff a xor b xor c xor d=0, an affine parallelogram in F2^8. [ENGINEERING; Tested]
BEA-25	Alignment	Parity linear functional	Global bit parity is a linear functional: parity(target)=parity(source) xor parity(delta). Equal parity does not imply equal Hamming weight or entropy. [RETAIN; Tested]
BEA-26	Hinge formalism	Commuting semantic hinge	A representation transform is semantically preserving when nu_t o H = nu_s on its declared domain. [ENGINEERING; Formal definition]
BEA-27	Hinge formalism	Partial invariant selector	A hinge certificate names exactly which signature coordinates are preserved and which break. [ENGINEERING; Implemented]
BEA-28	Hinge formalism	Non-invariant ledger	Homeomorphism, homotopy equivalence, linguistic universality and physical conservation remain explicit non-claims unless separately witnessed. [RETAIN; Certificate output]

ID	Domain	Operator	Definition / disposition / validation
BEA-29	Validation	Cross-profile falsification matrix	Candidate invariants are rerun across fonts, encodings, alignments, languages and tokenizers; one counterexample limits the claim. [RETAIN; Four-font data]
BEA-30	Validation	Profile-specificity conclusion	Failure outside one profile demonstrates profile dependence; it does not prove a unique language singularity. [CORRECTED; Claims ledger]
BEA-31	Referential core	BEA certificate record	The certificate stores profiles, values, boundary signatures, alignment witness, preserved/broken features, claim level and non-claims. [ENGINEERING; Implemented]
BEA-32	Evaluation	BEA phase order	Parse, normalize, resolve, transduce, evaluate semantics, align code, extract boundary, compare features, issue certificate, then continue support/compatibility/guard/lineage. [ENGINEERING; Schedule shipped]

B Source register and hashes

ID	Source	SHA-256 and role
SRC-A	Dutch-number dialogue Markdown	`7aa55d4f03e07c6ec058b1dd05554c03730d0f6263803b9364fe34ccce6c1851`. Earlier 3.6 source for radix, active-bit, Dutch number-order, geometry and claims-boundary analysis.
SRC-B	Unified Geometric-Topological Substrate (UGTS-0)	`c991aba24cc939ba680697b62b9e19545131cf34060793490bf149c6e2ab522e`. Query-first relation/event architecture, topology, lineage and evidence discipline.
SRC-C	UGTS-GN 1.1 GPU-native addendum	`3a6b6c57118ca4e2d7d7e577508c3477a095d133791f3708cb56a054285225d2`. Typed ABI, narrow one-bit semantics, precision, validation and claims ledger.
SRC-D	Pasted markdown(3).md	`04cbc3c4d4d791c4f7639ac9fc127d94fc3e4ecbc6cf20043aa56ff5b3eef1fc`. Source for the BEA extraction: proposed alias, alphabet geometry, glyph counters, leet map, ASCII XOR table and hinge discussion.

The raw uploaded sources are not copied into the ZIP. Privacy-safe normalized notes and hashes preserve traceability without redistributing unrelated dialogue content.

C Package layout

```
UGTS_KC_3_6_1_BEA_Tom_Klootwijk/
  README.md
  CHANGELOG.md
  NOTICE.md
  VERSION
  pyproject.toml
  requirements-analysis.txt
  validation_report.json
  report/UGTS_KC_3_6_1_BEA.pdf
  report/UGTS_KC_3_6_1_BEA.tex
  report/figures/*.png
  spec/BEA_FORMAL_DEFINITION.md
  spec/ugts_kc_3_6_1_bea.schema.json
  spec/operator_catalog_3_6_1_bea.{csv,json}
  spec/pattern_recipes_3_6_1_bea.{csv,json}
  spec/claims_ledger_3_6_1_bea.{csv,json}
  data/bea_xor_alignment.{csv,json}
  data/bea_symbol_paths.json
  data/bea_analysis/bea_glyph_topology_profiles.csv
  data/bea_analysis/masks/*.png
  examples/ugts_kc_3_6_1_bea_example.json
  examples/demo_bea.py
  examples/demo_bea_output.json
  src/ugts36/bea.py
  src/ugts36/*.py
  tests/test_ugts36.py
  tests/test_ugts361_bea.py
  tools/analyze_glyph_topology.py
  tools/generate_bea_figures.py
  tools/generate_bea_tex_tables.py
  sources/source_register.json
  sources/SRC_D_BEA_DISTILLATION.md
  checksums/SHA256SUMS.txt
```